

Firestore Authentication Notes (React CRA + Modular CSS)

Learning Objectives

- Connect a React app with Firestore.
- Configure Firestore Authentication.
- Create a Signup page.
- Create a Login page.
- Style using CSS Modules.
- Handle errors and success messages.

What is Firestore?

Firestore is a **Backend-as-a-Service (BaaS)** platform developed by **Google** that helps developers build web and mobile applications without creating and managing their own backend server.

Instead of building your own authentication system, database, storage, notifications, and hosting, Firestore provides these services out of the box.

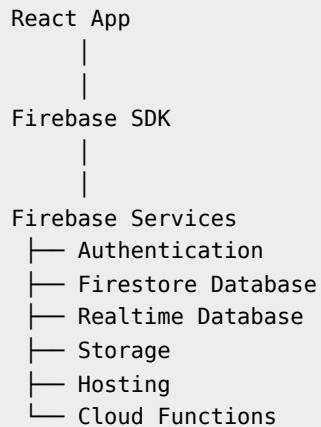
Why do we use Firestore?

Firestore is used to:

- Authenticate users (Login/Signup)
- Store application data
- Upload and store files (images, videos, PDFs)
- Host websites
- Send push notifications
- Analyse user behaviour

- Build applications faster without managing servers
-

How Firebase Works



Your React application communicates with Firebase through the Firebase SDK. Firebase processes the request and returns the required data.

Main Firebase Services

1. Authentication

Used for user login and signup.

Supports:

- Email & Password
- Google Login
- Facebook Login
- GitHub Login
- Phone Number Login

Example:

```
createUserWithEmailAndPassword(auth,email,password);
```

2. Firestore Database

A cloud NoSQL database used to store structured application data.

Example:

```
users |— user1 |— user2 |— user3
```

3. Realtime Database

Synchronises data instantly across all connected users.

Common use cases:

- Chat applications
- Live location
- Live dashboards
- IoT sensor data

4. Storage

Stores files such as:

- Images
- Videos
- PDFs
- Documents

Example:

```
Profile Picture  
Resume
```

Step 1: Create Firebase Project

1. Go to <https://console.firebase.google.com/>
2. Click **Create Project**
3. Enter project name.
4. Continue → Finish.

Step 2: Register Web App

- Click **</> Web**
- Enter app name.
- Click **Register App**.

Firebase provides:

```
const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "..."
};
```

Step 3: Install Firebase

```
npm install firebase
```

Step 4: Folder Structure

```
src
|
├─ firebase.js
├─ App.js
|
├─ pages
|   │   └─ Login.js
|   │   └─ Signup.js
|
├─ styles
|   │   └─ Login.module.css
|   │   └─ Signup.module.css
|
```

Step 5: Firebase Configuration

firebase.js

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseio.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT.appspot.com",
  messagingSenderId: "XXXXXXXX",
  appId: "XXXXXXXX"
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
```

Step 6: Enable Authentication

Firebase Console

Authentication

→ Sign-in Method

Enable

- Email/Password

Save.

Signup Page

Signup.js

```
import { useState } from "react";
import styles from "../styles/Signup.module.css";
import { auth } from "../firebase";

import { createUserWithEmailAndPassword } from "firebase/auth";

import Swal from "sweetalert2";

function Signup() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const signup = async (e) => {
    e.preventDefault();

    if (!email || !password) {
      Swal.fire({
        icon: "warning",
        title: "Missing Fields",
        text: "Please enter email and password."
      });
      return;
    }

    try {

      await createUserWithEmailAndPassword(
        auth,
        email,
        password
      );

      Swal.fire({
```

```

        icon: "success",
        title: "Account Created",
        text: "Signup Successful!"
    });

    setEmail("");
    setPassword("");

    } catch (error) {

        Swal.fire({
            icon: "error",
            title: "Signup Failed",
            text: error.message
        });

    }
};

return (

    <div className={styles.container}>

        <form className={styles.form} onSubmit={signup}>

            <h2>Create Account</h2>

            <input
                type="email"
                placeholder="Enter Email"
                value={email}
                onChange={(e)=>setEmail(e.target.value)}
            />

            <input
                type="password"
                placeholder="Enter Password"
                value={password}
                onChange={(e)=>setPassword(e.target.value)}
            />

            <button>Create Account</button>

        </form>

    </div>

);

}

```

```
export default Signup;
```

Signup.module.css

```
.container{
display:flex;
justify-content:center;
align-items:center;
height:100vh;
background:#f3f4f6;
}

.form{
width:350px;
padding:30px;
background:white;
border-radius:10px;
display:flex;
flex-direction:column;
gap:15px;
box-shadow:0 5px 15px rgba(0,0,0,.2);
}

.form h2{
text-align:center;
}

.form input{
padding:12px;
font-size:16px;
border:1px solid #ccc;
border-radius:5px;
}

.form button{
padding:12px;
border:none;
background:#2563eb;
color:white;
font-size:16px;
cursor:pointer;
border-radius:5px;
}

.form button:hover{
```



```
background:#1d4ed8;
}
```

Login Page

Login.js

```
import { useState } from "react";
import styles from "../styles/Login.module.css";

import { auth } from "../firebase";

import { signInWithEmailAndPassword } from "firebase/auth";

import Swal from "sweetalert2";

function Login() {

  const [email,setEmail]=useState("");
  const [password,setPassword]=useState("");

  const login = async(e)=>{

    e.preventDefault();

    if(!email || !password){

      Swal.fire({
        icon:"warning",
        title:"Missing Fields",
        text:"Please enter email and password."
      });

      return;
    }

    try{

      await signInWithEmailAndPassword(
        auth,
        email,
        password
      );

      Swal.fire({
```

```

        icon:"success",
        title:"Welcome",
        text:"Login Successful!"
    });

    setEmail("");
    setPassword("");

}

catch(error){

    Swal.fire({
        icon:"error",
        title:"Login Failed",
        text:error.message
    });

}

}

return(

<div className={styles.container}>

<form className={styles.form} onSubmit={login}>

<h2>Login</h2>

<input
type="email"
placeholder="Enter Email"
value={email}
onChange={(e)=>setEmail(e.target.value)}
/>

<input
type="password"
placeholder="Enter Password"
value={password}
onChange={(e)=>setPassword(e.target.value)}
/>

<button>Login</button>

</form>

</div>

);

```

```
}  
  
export default Login;
```

Login.module.css

```
.container{  
  display:flex;  
  justify-content:center;  
  align-items:center;  
  height:100vh;  
  background:#eef2ff;  
}  
  
.form{  
  width:350px;  
  padding:30px;  
  background:white;  
  border-radius:10px;  
  display:flex;  
  flex-direction:column;  
  gap:15px;  
  box-shadow:0 5px 15px rgba(0,0,0,.2);  
}  
  
.form h2{  
  text-align:center;  
}  
  
.form input{  
  padding:12px;  
  font-size:16px;  
  border:1px solid #ccc;  
  border-radius:5px;  
}  
  
.form button{  
  padding:12px;  
  border:none;  
  background:#16a34a;  
  color:white;  
  font-size:16px;  
  cursor:pointer;  
  border-radius:5px;  
}  
  
.form button:hover{
```

```
background:#15803d;
}
```

App.js

```
import Signup from "../pages/Signup";
// import Login from "../pages/Login";

function App() {
  return (
    <div>
      <Signup />
      { /* <Login /> */ }
    </div>
  );
}

export default App;
```

Important Firebase Authentication Functions

Function	Purpose
<code>initializeApp()</code>	Initializes the Firebase application.
<code>getAuth()</code>	Creates the Authentication instance.
<code>createUserWithEmailAndPassword()</code>	Registers a new user with email and password.
<code>signInWithEmailAndPassword()</code>	Logs in an existing user.
<code>signOut()</code>	Logs out the current user.
<code>onAuthStateChanged()</code>	Detects authentication state changes.

Common Errors

Error	Reason
auth/email-already-in-use	Email already exists.
auth/invalid-email	Invalid email format.
auth/weak-password	Password is less than 6 characters.
auth/user-not-found	User account does not exist.
auth/wrong-password	Incorrect password.
auth/network-request-failed	Internet connection problem.